

Unitree Go1 Obstacle Avoidance

Projektpraktikum – Systematische Analyse der Sicherheit in Embedded und AI-basierten Systemen

Ridwan Islam, Mateo Tabak, Ahmet Yetisir & Vincent Rauscher
September 23, 2025



TECHNISCHE
UNIVERSITÄT
DARMSTADT

CYSEC

Contents

1	Introduction	3
1.1	Goal	3
1.2	Herangehensweise	3
2	Getting Started	4
2.1	Starting the Robot	4
2.2	Shutting Down the Robot	4
3	General Config	4
3.1	Robot Configuration	4
3.2	LiDAR Configuration	5
3.3	Laptop Configuration	5
4	Einrichtung der Entwicklungsumgebung	6
4.1	Ubuntu Laptop vorbereiten	6
4.2	Installation von ROS Noetic	6
5	General Setup	6
5.1	Hardware Setup	6
5.2	Software Setup	7
6	Steuerung mit Raspberry Pi	7
7	LiDAR Integration	7
8	Lokalisierung und Kartierung	7
8.1	Terminal-Setup	8
8.2	Map-Speicherung	9
9	Probleme mit AMCL	9
10	Implementierung: Lokale Navigation mit reaktiver Umfahrung, Corner-Stop und dynamischem Drift	9
10.1	Ziel & Grundprinzip	9
10.2	Sensorik und Ausgänge	9
10.3	Sektorbasierte Auswertung	9
10.4	Seitennachführung und Glättung	10
10.5	Reaktive Umfahrung	10
10.6	Corner-Stop	10
10.7	Dynamischer Drift	10
10.8	Ausführung der Skripte	10
10.9	Ergebnis	11
11	Hinweise	11

1 Introduction

1.1 Goal

Ziel des Versuchs war es, den Roboter Go1 auf eine festgelegte Route zu schicken und dabei auftretende Hindernisse mittels Obstacle Avoidance durch das Auslesen des LiDAR-Sensors zu umgehen.

1.2 Herangehensweise

Für die Umsetzung wurden zwei Ansätze betrachtet:

- **Ansatz 1: Manuelles Codieren**

Die Route wurde manuell vorgegeben und das Ausweichen von Hindernissen durch direktes Auslesen und Auswerten der LiDAR-Daten programmiert. So konnte der Roboter einer definierten Strecke folgen und gleichzeitig dynamisch Hindernissen ausweichen.

- **Ansatz 2: SLAM-basierte Navigation**

Mithilfe von HectorSLAM sollte zunächst eine Karte der Umgebung erstellt werden. Darauf aufbauend war die Lokalisierung des Roboters auf der Karte geplant, sowie die anschließende Routenplanung vom Startpunkt zum Zielpunkt. Während der Fahrt sollten neu auftretende Hindernisse durch Obstacle Avoidance umgangen werden. Für diesen Ansatz war zunächst vorgesehen, den Raspberry Pi als zentrale Plattform einzusetzen. Dabei sollte der Pi per WLAN mit dem Laptop verbunden sein, während LiDAR und Roboter über LAN-Adapter angebunden wurden. In der Praxis zeigte sich jedoch, dass diese Konfiguration erhebliche Probleme verursachte (siehe Abschnitt ??). Auch nachdem wir den Ansatz ohne den Raspberry Pi direkt mit dem Laptop verfolgten, konnten wir die SLAM-basierte Navigation nicht erfolgreich umsetzen, da zusätzliche softwareseitige Schwierigkeiten auftraten.

Letztlich wurde **Ansatz 1** erfolgreich umgesetzt. Bei **Ansatz 2** traten sowohl hardware- als auch softwareseitige Hürden auf, die eine vollständige Implementierung im gegebenen Zeitrahmen verhinderten. Dieser Bericht dokumentiert den Entwicklungsprozess, die aufgetretenen Herausforderungen und die finale, funktionierende Implementierung der lokalen Hindernisumfahrung.¹

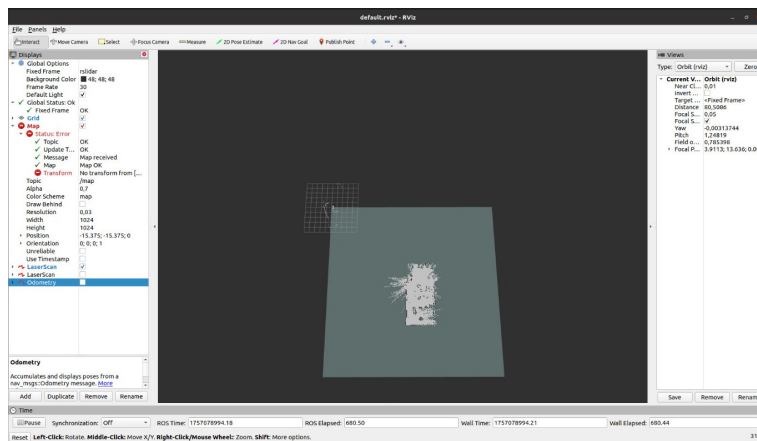


Figure 1: SLAM Map des Student-Labs

¹Für die Erstellung dieses Dokuments wurde unterstützend ein KI-Sprachmodell verwendet.

2 Getting Started

2.1 Starting the Robot

- **Startup Pose:** Vor dem Einschalten muss der Roboter korrekt positioniert werden. Er wird flach auf den Bauch gelegt, die Beine seitlich ausgestreckt, sodass Füße und Knie den Boden berühren (siehe Abbildung 2).
- **Powering On the Robot Dog:** Zum Einschalten den Power-Button am Akku doppelklicken. Beim zweiten Klick den Knopf gedrückt halten, bis die grünen LEDs leuchten. Der Bootvorgang dauert etwa 1–2 Minuten; danach richtet sich der Roboter automatisch auf.
- **Powering On the Remote Control:** Auch die Fernbedienung wird mit einem Doppelklick gestartet. Beim zweiten Klick den Button ca. 5 Sekunden gedrückt halten, bis ein Bestätigungston ertönt. Die Fernbedienung verbindet sich automatisch mit dem Roboter.

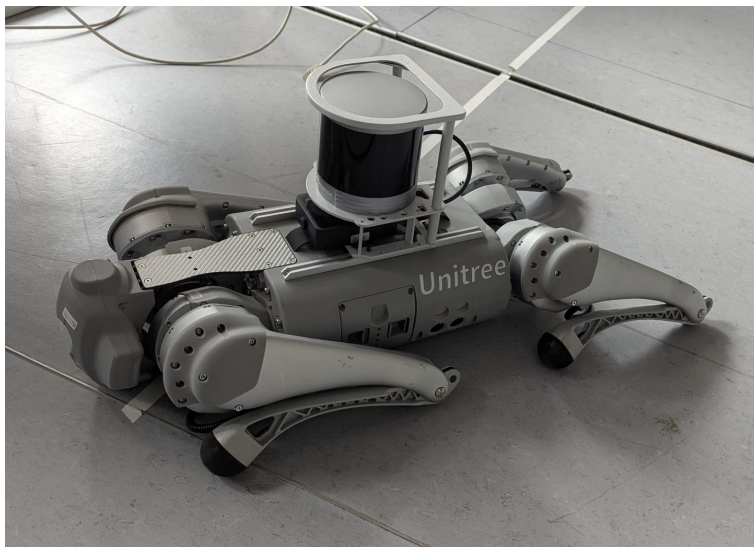


Figure 2: Startposition des Unitree Go1 vor dem Einschalten

2.2 Shutting Down the Robot

- **Laying Pose:** Stelle sicher, dass der Roboter stillsteht. Halte anschließend die L2-Taste gedrückt, drücke zweimal die A-Taste, danach die B-Taste, und lasse zuletzt die L2-Taste los. Kurzform: **HOLD L2 + A + A + B, RELEASE L2**. Der Roboter legt sich nun in die gleiche Position wie beim Start.
- **Powering Off the Robot Dog:** Zum Ausschalten den Power-Button am Akku doppelklicken. Beim zweiten Klick gedrückt halten, bis die grünen LEDs erlöschen.
- **Powering Off the Remote Control:** Zum Ausschalten der Fernbedienung den Power-Button doppelklicken, beim zweiten Klick ca. 5 Sekunden gedrückt halten, bis der Bestätigungston erklingt und die Status-LED erlischt.

3 General Config

3.1 Robot Configuration

- Modell: Unitree Go1 EDU
- Schnittstellen: 1 × LAN-Port, WLAN-Hotspot, USB-C, LiDAR Stromanschluss

- Steuerung: High-Level-Control über ROS (unitree_legged_sdk)
- IP-Adresse des Roboters:
 - LAN: 192.168.123.161
 - WLAN-Hotspot: 192.168.12.107
- WLAN-Verbindung:
 - Nach dem Booten startet ein Access Point Unitree_GoXXXXXX.
 - Passwort: 00000000.
- Kommunikationsprotokoll: UDP über ROS-Bridge

3.2 LiDAR Configuration

- Modell: RoboSense RS-Helios 16P
- Standard-IP des Sensors: 192.168.1.200
- Erwartete Host-IP (Laptop/Pi): 192.168.1.210
- Stromanschluss am Go1 (siehe Abbildung 3)
- Datenports:
 - MSOP (Messdaten): UDP-Port 6699
 - DIFOP (Konfiguration): UDP-Port 7788



Figure 3: Stromversorgung des LiDAR am Go1

3.3 Laptop Configuration

- Betriebssystem: Ubuntu 20.04 LTS (64bit) <https://www.releases.ubuntu.com/focal/>
- Netzwerk:
 - WLAN: Verbindung zum Go1-Hotspot (192.168.12.x)
 - Ethernet/USB-LAN: Verbindung zum LiDAR (192.168.1.210)
- ROS-Version: ROS Noetic <https://wiki.ros.org/noetic/Installation/Ubuntu>
- Workspace: ~/catkin_ws

4 Einrichtung der Entwicklungsumgebung

4.1 Ubuntu Laptop vorbereiten

Zunächst testeten wir eine **virtuelle Maschine (VM)** unter Windows/Mac. Durch instabiles WLAN, falsch geroutete Ports und IP-Konflikte war die Kommunikation mit Roboter und LiDAR unzuverlässig. Wir sind daher auf eine **native Ubuntu-Installation** gewechselt (stabiler, ROS-Support out-of-the-box).

Nach der Grundinstallation des Systems wurden zunächst alle Pakete aktualisiert:

```
sudo apt update && sudo apt -y upgrade
```

Zusätzlich sind Basis-Werkzeuge für Kompilierung und Versionsverwaltung notwendig:

```
sudo apt -y install build-essential git cmake \  
python3-pip python3-rosdep python3-rosinstall \  
python3-vcstools curl
```

4.2 Installation von ROS Noetic

ROS-Repository einbinden und Schlüssel importieren:

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu focal main" \  
> /etc/apt/sources.list.d/ros1-latest.list'
```

```
curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc \  
| sudo apt-key add -
```

Anschließend die Desktop-Variante installieren:

```
sudo apt update  
sudo apt -y install ros-noetic-desktop-full
```

Damit ROS in jeder Shell verfügbar ist, die Setup-Datei in die `.bashrc` aufnehmen:

```
echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc  
source ~/.bashrc
```

5 General Setup

5.1 Hardware Setup

Der Unitree Go1 EDU verfügt über folgende Schnittstellen:

- **LAN-Port** – für kabelgebundene Kommunikation
- **WLAN-Hotspot** – SSID Unitree_GoXXXXXXX, IP-Adresse des Roboters: 192.168.12.107
- **USB-C** – für externe Erweiterungen

Für die Versuche wurde der Laptop parallel verbunden:

- Über **WLAN** mit dem Go1-Hotspot
- Über **USB-LAN-Adapter** mit dem LiDAR-Sensor

Die statischen IP-Adressen wurden wie folgt gesetzt:

- LiDAR (RS-Helios 16P): 192.168.1.200
- Host (Laptop): 192.168.1.210

Damit ergibt sich eine Topologie, bei der der Laptop gleichzeitig per WLAN mit dem Go1 und per LAN mit dem LiDAR verbunden ist.

5.2 Software Setup

Als Betriebssystem wurde **Ubuntu 20.04 LTS (64-bit)** verwendet, da diese Version die stabilste Unterstützung für **ROS Noetic** bietet. Die Entwicklungsumgebung wurde in einem Catkin-Workspace (`~/catkin_ws`) eingerichtet. Darin wurden folgende ROS-Pakete installiert:

- `rslidar_sdk` – Treiberpaket für den RS-Helios LiDAR
- `pointcloud_to_laserscan` – Konvertierung von Punktwolken in 2D-Laserscans
- `hector_slam` – 2D-SLAM zur Kartenerstellung
- `unitree_legged_sdk` – ROS-Integration des Go1

6 Steuerung mit Raspberry Pi

Steuerung des Roboters mit dem Raspberry Pi als zentraler Recheneinheit

- **Headless-Start scheitert:** `rslidar_sdk` verlangte beim Start eine aktive GUI/Display-Session; im Headless-Betrieb lieferte der Sensor keine gültigen Daten.
- **Netzwerk-kette instabil:** Pi per WLAN zum Laptop, Roboter und LiDAR per LAN→USB am Pi. Der USB-LAN-Adapter wurde nicht zuverlässig erkannt, die Schnittstelle lieferte kein gültiges Signal.

Daher Umstieg auf einen Laptop mit nativem Ubuntu 20.04 und ROS Noetic.

7 LiDAR Integration

Der RS-Helios 16P wurde über UDP in ROS eingebunden. Die Kommunikation nutzt zwei Ports:

- **MSOP (Messdaten):** UDP-Port 6699
- **DIFOP (Konfiguration):** UDP-Port 7788

Die Netzwerkkonfiguration erfolgte über die Datei `config.yaml` im `rslidar_sdk`. Nach erfolgreicher Anpassung konnten die Punktwolken in **RViz** visualisiert werden.

Durch das Node `pointcloud_to_laserscan` wurde die Punktwolke auf eine 2D-Ebene projiziert, sodass ein Laser-Scan als Topic `/scan` verfügbar war. Dieses Topic bildete die Grundlage für die weiteren SLAM- und Navigationsaufgaben.

8 Lokalisierung und Kartierung

Für die Kartierung der Umgebung wurde das **Hector SLAM**-Paket eingesetzt. Die von `pointcloud_to_laserscan` erzeugten 2D-Laserscans dienten dabei als primäre Eingabe. Ein wesentlicher Vorteil von Hector SLAM besteht darin, dass es **keine Odometrie-Daten benötigt**, sondern sich ausschließlich auf die Scan-Daten stützt. Dadurch eignet es sich besonders für Systeme ohne hochpräzise Bewegungssensorik.

Die Vorgehensweise gliederte sich in folgende Schritte:

1. Start des LiDAR-Treibers mittels `rslidar_sdk`.
2. Umwandlung der Roh-Punktwolken in 2D-Laserscans durch `pointcloud_to_laserscan`.
3. Publikation der notwendigen TF-Transformation zwischen `base_link` und `rslidar`.
4. Ausführung des `hector_mapping`-Nodes unter Nutzung der Topics:
 - `/scan` (Laser-Scans)
 - `/map` (erzeugte Karte)
 - `/odom` (geschätzte Pose)

Mit diesem Setup konnte eine **2D-Karte** der Umgebung in Echtzeit erstellt werden. Die Kartierung wurde in **RViz** visualisiert, während der Roboter manuell durch die Umgebung bewegt wurde. Dies ermöglichte eine kontinuierliche Beobachtung der Scan-Daten und der entstehenden Karte.

8.2 Map-Speicherung

Nach Abschluss der Kartierung konnte die erzeugte Karte gespeichert werden:

```
source /opt/ros/noetic/setup.bash
mkdir -p ~/maps
roslaunch map_server map_saver -f ~/maps/gang_hector
# erzeugt: ~/maps/gang_hector.pgm + ~/maps/gang_hector.yaml
```

Die resultierende Karte liegt im Format `.pgm` (Bild) und `.yaml` (Metadaten) vor und kann in späteren Sitzungen wieder in ROS geladen werden.

9 Probleme mit AMCL

Bei der Verwendung von **AMCL** (Adaptive Monte Carlo Localization) trat das Problem auf, dass der notwendige Transform `map` $\rightarrow$ `odom` nicht veröffentlicht werden konnte. Dies lag daran, dass mehrere Grundvoraussetzungen gleichzeitig nicht erfüllt waren:

1. **Fehlerhafte Laser-Scans:** Die Scans wurden anfangs auf einem falschen Topic veröffentlicht bzw. besaßen unplausible Zeitstempel (z. B. sehr alte Zeit). Dadurch konnte der `tf`-Baum die Daten nicht korrekt in den Scan-Zeitpunkt transformieren. Dieses Problem wurde durch das Nachschalten von `/scan_retime` behoben.
2. **Fehlende Odometrie:** Es existierte keine kontinuierlich aktualisierte Transformation `odom` $\rightarrow$ `base_link`. Ohne diese Bewegungsinformation können die AMCL-Partikel nicht fortgeschrieben werden, was zur Folge hat, dass AMCL praktisch „stehenbleibt“.
3. **Keine Initialpose:** In RViz wurde keine Startpose (*2D Pose Estimate*) gesetzt. Dadurch war AMCL nicht in der Lage, die Ausgangslage des Roboters sinnvoll zu schätzen.

Das Resultat dieser Probleme war, dass im `tf`-Baum kein `map`-Frame existierte. Daraufhin wartete `move_base` vergeblich auf eine gültige Transformation zwischen `map` und `base_link`.

10 Implementierung: Lokale Navigation mit reaktiver Umfahrung, Corner-Stop und dynamischem Drift

10.1 Ziel & Grundprinzip

Die Navigation des Go1 basiert auf einem rein lokalen Verfahren ohne globale Karte oder SLAM. Der Roboter hält sich möglichst mittig im Korridor, steuert ausschließlich die Geschwindigkeiten v_x (vorwärts) und v_y (seitwärts) und fixiert die Gierachse (yaw) konstant auf 0. Hindernisse werden durch LiDAR-Daten erkannt und reaktiv umfahren, während an echten Korridorecken ein gezielter Stop erfolgt.

10.2 Sensorik und Ausgänge

- **Eingänge:** Zwei Laser-TTopics (`/scan` und `/scan_low`) werden ausgewertet. Für die Frontdetektion wird konservativ die kleinere Distanz aus beiden Scans genutzt, um auch niedrige Objekte sicher zu erkennen.
- **Ausgänge:** Steuerbefehle (v_x , v_y) werden über die Unitree-SDK per UDP an den Roboter gesendet.

10.3 Sektorbasierte Auswertung

Die Laserscans werden in mehrere Sektoren unterteilt:

- **Front:** $\pm 30^\circ$ um die Roboterachse zur Erkennung freier Strecke.
- **Links/Rechts:** Jeweils $\approx 90^\circ$ zur Wandabstandsschätzung.
- **Diagonale Sektoren:** Zur frühzeitigen Hinderniserkennung und Entscheidungsfindung.

Zur Rauschunterdrückung werden Quantile (z. B. 20 %) statt Minimalwerte genutzt, um einzelne Ausreißer zu dämpfen.

10.4 Seitennachführung und Glättung

Im Normalbetrieb hält ein Wandfolger den Roboter in der Mitte des Korridors. Aus der Differenz der Seitenabstände (d_L , d_R) wird ein Center-Error berechnet, der über einen gleitenden Mittelwert gefiltert wird. Daraus entsteht eine sanfte Korrekturgeschwindigkeit v_y , während v_x über eine Rampe hochläuft. Eine Hysterese stellt sicher, dass der Roboter nur dann weiterfährt, wenn der Frontbereich für kurze Zeit stabil frei ist.

10.5 Reaktive Umfahrung

Die Umfahrung ist zweistufig und in den Normalmodus integriert:

1. **Vorwarn-Bias:** Wenn sich der Frontbereich langsam schließt, wird ein kleiner seitlicher Bias in Richtung der freieren Seite hinzugefügt und v_x leicht reduziert.
2. **Nahbereichs-Reaktion:** Bei engem Abstand wird v_x stark begrenzt und eine gezielte seitliche Bewegung (v_y) ausgelöst, bis der Frontbereich wieder frei ist.

10.6 Corner-Stop

Um eine Gang-Ecke zuverlässig zu erkennen, ohne Türen fälschlich als solche zu interpretieren, werden drei Bedingungen kombiniert:

1. Der Frontbereich schließt sich deutlich (Ende des Gangs).
2. Auf der rechten Seite bleibt eine Wandstruktur erkennbar.
3. Links öffnet sich ein breiter, zusammenhängender Bereich.

Sind alle Kriterien erfüllt, stoppt der Roboter gezielt (**GOAL**-Zustand).

10.7 Dynamischer Drift

Während der Tests zeigte der Go1 einen **dynamischen Seitendrift**, dessen Richtung und Stärke von Parametern wie **Schrittgröße**, **Geschwindigkeit** und insbesondere dem **Akkustand** abhängen. Mit sinkender Batteriespannung verstärkte sich der Drift meist in eine feste Richtung (leicht rechts- oder linksseitig). Da der Roboter yaw=0 hält, wirkt sich jede Asymmetrie in der Beinkraft oder Mechanik direkt auf die laterale Bewegung aus.

Kompensation. Der Drift wurde durch Anpassung der Reglerparameter ausgeglichen:

- Asymmetrische Zielabstände (`left_target/right_target`) entgegengesetzt der Driftrichtung.
- Reduzierte Reglerverstärkung (`k_center`) und vergrößertes `deadband_center` für sanftere Korrekturen.
- Optionale lineare Korrektur abhängig von der Batteriespannung.

Insgesamt konnte so trotz der Hardwareasymmetrien eine stabile, gerade Korridorfahrt erreicht werden.

10.8 Ausführung der Skripte

Die Ausführung ist im `README.md` beschrieben. Vor jedem Start muss die ROS-Umgebung geladen und ins Beispielverzeichnis gewechselt werden:

```
source /opt/ros/noetic/setup.bash
cd ~/unitree_legged_sdk/example_py
```

Demo 1 – Corner / Gap-Aware Navigation

```
python3 go1_corridor_bypass_corner_pausealign_gapaware2.py ...
```

Dieses Skript realisiert die vollständige autonome Korridornavigation: Der Roboter hält sich selbstständig mittig, erkennt Hindernisse und Ecken, umfährt diese seitlich und richtet sich danach wieder in der Korridormitte aus. Die wichtigsten Parameter steuern Geschwindigkeiten, Abstände und die automatische Ausrichtung.

Demo 2 – Inverse-Center & Diagnostic Watch

```
python3 go1_corridor_bypass_invcenter_diagwatch_ultra_lowguard.py ...
```

Eine vereinfachte Variante mit versetzter (inverser) Zentrierung und aktiver LiDAR-Diagnose. Sie dient zur schnellen Überprüfung der Sensorik, der Stabilität des Reglers und der Datenkonsistenz zwischen `/scan` und `/scan_low`.

10.9 Ergebnis

Beide Skripte ermöglichen eine stabile, lokale Navigation ohne SLAM oder globale Planung.

- **Demo 1** zeigt die komplette Obstacle-Avoidance inklusive Eckenerkennung, Re-Zentrierung und sanfter Geschwindigkeitsrampen.
- **Demo 2** bietet eine minimalistische Testumgebung zur Validierung der LiDAR-Daten und der Driftkompensation.

11 Hinweise

Im Verlauf der Arbeit traten verschiedene wiederkehrende Probleme auf, die teilweise technischer, teilweise systembedingter Natur waren. Die wichtigsten Punkte und Lösungsansätze sind im Folgenden zusammengefasst:

- **Problem: Roboter zögert oder bleibt vor Hindernissen stehen.**
 - **Ursache:** Sicherheitsschwellen sind zu konservativ (er wartet auf eine zu große Lücke) oder der bodennahe Scan `/scan_low` erkennt ein flaches, für den Hauptsensor unsichtbares Hindernis.
 - **Lösung:** Anpassung der Parameter `stop_thresh` und `resume_thresh` sowie Gewichtung des bodennahen Scans reduzieren.
- **Problem: Nervöses Zick-Zack-Fahren im Korridor.**
 - **Ursache:** Zu aggressive Regelung für das Mittehalten oder zu starkes Rauschen in den LiDAR-Daten.
 - **Lösung:** Totbereich (`deadband_center`) vergrößern, Filterung (`alpha_center_long`) erhöhen, um Reaktionen zu glätten.
- **Problem: Roboter fährt zu nah an Wänden oder Hindernissen vorbei.**
 - **Ursache:** Sicherheitsabstände (`left_target/right_target`) zu klein oder unzureichende Kompensation des **dynamischen Drifts**.
 - **Lösung:** Zielabstände asymmetrisch anpassen, Reglerverstärkung (`k_center`) reduzieren und Drift über Batteriespannung bzw. Schrittparameter kompensieren.
- **Problem: Dynamischer Drift.**
 - **Ursache:** Mit sinkender Akkuspannung, variierender Schrittgröße oder Geschwindigkeit neigt der Go1 dazu, seitlich zu drift (typischerweise nach rechts).
 - **Lösung:** Reglerparameter dynamisch anpassen oder Drift mit festen Korrekturwerten (Offset im v_y) kompensieren.
- **Problem: Frontkamera liefert keine Bilddaten.**
 - **Ursache:** Die Frontkamera des Go1 war während der Tests defekt und lieferte keine Bildsignale.

-
- **Lösung:** Nutzung der Kamera war im Rahmen dieses Projekts nicht möglich. Für zukünftige Arbeiten sollte die Hardware geprüft oder eine externe Kamera (z. B. USB-Kamera mit eigener ROS-Node) integriert werden.

- **Problem: Fehlende oder fehlerhafte Odometrie.**

- **Ursache:** Der Roboter übermittelt keine kontinuierliche `odom→base_link`-Transformation, wodurch AMCL und andere Lokalisierungsverfahren nicht funktionieren.
- **Lösung:** Eigene Odometrie-Schätzung implementieren (z. B. über IMU und Velocity-Integration) oder alternative SLAM-Methoden wie GMapping, Cartographer oder RTAB-Map testen.

- **Problem: Lokalisierung in der Karte nicht stabil.**

- **Ursache:** Kombination aus fehlender Odometrie, zeitlich fehlerhaften Laser-Scans und unpräzisen TF-Transformationen (`map→odom`).
- **Lösung:** Vor Nutzung von AMCL die TF-Kette vollständig prüfen (`tf_view_frames`) und gegebenenfalls eigene Transformations-Node implementieren.

- **Problem: Instabile Kommunikation bei Verwendung einer virtuellen Maschine.**

- **Ursache:** ROS-Topics und Netzwerk-Interfaces werden durch die VM nicht korrekt weitergeleitet; Ports und IPs (z. B. UDP 6699, 7788) werden blockiert oder falsch geroutet.
- **Lösung:** Native Ubuntu-Installation auf Laptop verwenden; bei VM-Nutzung sicherstellen, dass Bridged Adapter aktiviert ist und alle UDP-Ports offen sind.

Diese Probleme traten in unterschiedlicher Kombination auf und zeigen, dass eine stabile Navigation nur durch korrekt konfiguriertes Netzwerk, robuste Sensorfusion und fein abgestimmte Reglerparameter gewährleistet werden kann.